

14_evaluation_detection_wGenes

August 31, 2026

```
[1]: library(Seurat)
# library(SeuratDisk)

library(reticulate)
library(anndata)

library(ggplot2)
library(ggpubr)
library(pheatmap)
library(dplyr)
library(tidyr)
library(RColorBrewer)
library(clustree)
library(repr)
library(ggdist)
library(ggbridges)
options(repr.plot.width=10, repr.plot.height=8)

library(UpSetR)
library(grid)

library(PRRoc)
library(Matrix)

library(reshape2)

library(GenomicRanges)
library(GenomicFeatures)

library(purrr)

library(scales)

getwd()

dataset_id <- "simulated_mm_RA_wGenes"
```

```

genome_id <- "mm10"
samples <- c("all")

dir.create("figures")
for (sample in samples) {
  dir.create(paste0("figures/figures_", dataset_id, "_", sample))
}
dir.create(paste0("figures/figures_", dataset_id))

colorTools <- c( # "MATES"="#F98A7B",
  "STARsolo_TE" = "#FFBC81",
  "STARsolo_TE_EM" = "#F3E088",
  "SoloTE_unique" = "#A4DD9B",
  "SoloTE_thr2" = "#6FC69D",
  "SoloTE_thr1" = "#4BB2BB",
  "SoloTE_thr0" = "#3989BF",
  "Stellarscope" = "#4F5D93",
  "simulated" = "grey70"
)

thrMinCells <- 500 * 0.05

```

Loading required package: SeuratObject

Loading required package: sp

Attaching package: 'SeuratObject'

The following objects are masked from 'package:base':

intersect, t

Attaching package: 'anndata'

The following object is masked from 'package:SeuratObject':

Layers

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

Loading required package: ggraph

Attaching package: 'ggraph'

The following object is masked from 'package:sp':

geometry

Attaching package: 'ggridges'

The following objects are masked from 'package:ggdist':

scale_point_color_continuous, scale_point_color_discrete,
scale_point_colour_continuous, scale_point_colour_discrete,
scale_point_fill_continuous, scale_point_fill_discrete,
scale_point_size_continuous

Loading required package: rlang

Attaching package: 'rlang'

The following object is masked from 'package:ggdist':

ll

Attaching package: 'Matrix'

The following objects are masked from 'package:tidyr':

expand, pack, unpack

Attaching package: 'reshape2'

The following object is masked from 'package:tidyr':

smiths

Loading required package: stats4

Loading required package: BiocGenerics

Attaching package: 'BiocGenerics'

The following objects are masked from 'package:dplyr':

combine, intersect, setdiff, union

The following object is masked from 'package:SeuratObject':

intersect

The following objects are masked from 'package:stats':

IQR, mad, sd, var, xtabs

The following objects are masked from 'package:base':

anyDuplicated, aperm, append, as.data.frame, basename, cbind,
colnames, dirname, do.call, duplicated, eval, evalq, Filter, Find,
get, grep, grepl, intersect, is.unsorted, lapply, Map, mapply,
match, mget, order, paste, pmax, pmax.int, pmin, pmin.int,
Position, rank, rbind, Reduce, rownames, sapply, setdiff, sort,
table, tapply, union, unique, unsplit, which.max, which.min

Loading required package: S4Vectors

Attaching package: 'S4Vectors'

The following objects are masked from 'package:Matrix':

expand, unname

The following object is masked from 'package:tidyr':

expand

The following objects are masked from 'package:dplyr':

first, rename

The following object is masked from 'package:utils':

findMatches

The following objects are masked from 'package:base':

expand.grid, I, unname

Loading required package: IRanges

Attaching package: 'IRanges'

The following objects are masked from 'package:dplyr':

collapse, desc, slice

The following object is masked from 'package:sp':

%over%

Loading required package: GenomeInfoDb

Loading required package: AnnotationDbi

Loading required package: Biobase

Welcome to Bioconductor

Vignettes contain introductory material; view with
'browseVignettes()'. To cite Bioconductor, see
'citation("Biobase")', and for packages 'citation("pkgname")'.

Attaching package: 'Biobase'

The following object is masked from 'package:rlang':

exprs

Attaching package: 'AnnotationDbi'

The following object is masked from 'package:dplyr':

select

Attaching package: 'purrr'

The following object is masked from 'package:GenomicRanges':

reduce

The following object is masked from 'package:IRanges':

reduce

The following objects are masked from 'package:rlang':

%@%, flatten, flatten_chr, flatten_dbl, flatten_int, flatten_lgl,
flatten_raw, invoke, splice

Attaching package: ‘scales’

The following object is masked from ‘package:purrr’:

discard

’/mnt/transfer/TEbenchmarking/Rscripts’

Warning message in dir.create("figures"):

“'figures' already exists”

Warning message in dir.create(paste0("figures/figures_", dataset_id, "_", sample)):

“'figures/figures_simulated_mm_RA_wGenes_all' already exists”

Warning message in dir.create(paste0("figures/figures_", dataset_id)):

“'figures/figures_simulated_mm_RA_wGenes' already exists”

```
[ ]: load("workspaces/13_evaluation_objectCreation_wGenes.Rdata")
```

1 Evaluation

1.1 Upset plots of detected TEs and TPs

1.1.1 Upsets of detected TEs

```
[3]: options(repr.plot.width=12, repr.plot.height=7.6)

for (sample in samples){

  print(sample)

  TPlist <- list()
  FNlist <- list()
  FPlist <- list()

  detectedList <- list()

  for (tool in c(names(objList), "simulated")){

    print(tool)
    if (tool == "simulated") {
      obj <- splatter_objs[[sample]]
    }else {
      obj <- objList[[tool]][[sample]]
    }
  }
}
```

```

    }

    detectedList[[sub("^(.*)_.*$", "\\1", obj@project.name)]] <-
    ↪Features(obj)
    TPlist[[sub("^(.*)_.*$", "\\1", obj@project.name)]] <-
    ↪intersect(Features(splatter_objs[[sample]]), Features(obj))
    FNlist[[sub("^(.*)_.*$", "\\1", obj@project.name)]] <-
    ↪setdiff(Features(splatter_objs[[sample]]), Features(obj))
    FPlist[[sub("^(.*)_.*$", "\\1", obj@project.name)]] <-
    ↪setdiff(Features(obj), Features(splatter_objs[[sample]]))
  }

  #pdf(file=paste0("figures/figures_", dataset_id, "_", sample, "/"
  ↪upset_detectedTEs.pdf"), width = 12, height = 7.6)
  show(UpSetR::upset(fromList(detectedList), nintersects = 15,
    sets=names(colorTools), keep.order = T, sets.bar.color=colorTools,
    text.scale = c(2, 2, 2, 1.65, 2.5, 1.5),
    order.by=c("freq"),
    point.size=3, mb.ratio = c(0.5, 0.5),
    set_size.show=F, set_size.scale_max=
    ↪max(lengths(detectedList))+1000, #show.numbers = F,
    nsets=length(objList)+1,
    sets.x.label="N. detected loci",
    mainbar.y.label="Intersection size")

  )
  grid.text(paste0("Detected TEs - ", sample ),x = 0.65, y=0.95,
  ↪gp=gpar(fontsize=18))
  #dev.off()

  #pdf(file=paste0("figures/figures_", dataset_id, "_", sample, "/"
  ↪upset_detected_TP.pdf"), width = 11, height = 7.5)
  show(UpSetR::upset(fromList(TPlist), nintersects = 15,
    sets=names(colorTools), keep.order = T, sets.bar.color=colorTools,
    text.scale = c(2, 2, 2, 1.65, 2.2, 1.5),
    order.by=c("freq"),
    point.size=3, mb.ratio = c(0.5, 0.5),
    set_size.show=F, set_size.scale_max= max(lengths(TPlist))+1000,
    ↪#show.numbers = F,
    nsets=length(c(objList, splatter_obj)),
    sets.x.label="N. correctly detected loci",
    mainbar.y.label="Intersection size")

  )
  grid.text(paste0("TP TEs - ", sample ),x = 0.65, y=0.95,
  ↪gp=gpar(fontsize=18))

```



```

show(UpSetR::upset(fromList(FPlist), nintersects = 15,
  sets=names(colorTools), keep.order = T, sets.bar.color=colorTools,
  text.scale = c(2, 2, 2, 1.65, 2.2, 1.5),
  order.by=c("freq"),
  point.size=3, mb.ratio = c(0.5, 0.5),
  set_size.show=F, set_size.scale_max= max(lengths(FPlist))+1000,
  ↪#show.numbers = F,
  nsets=length(c(objList, splatter_obj)),
  sets.x.label="N. FP",
  mainbar.y.label="Intersection size")
)
grid.text(paste0("FP TEs - ", sample ),x = 0.65, y=0.95,
  ↪gp=gpar(fontsize=18))

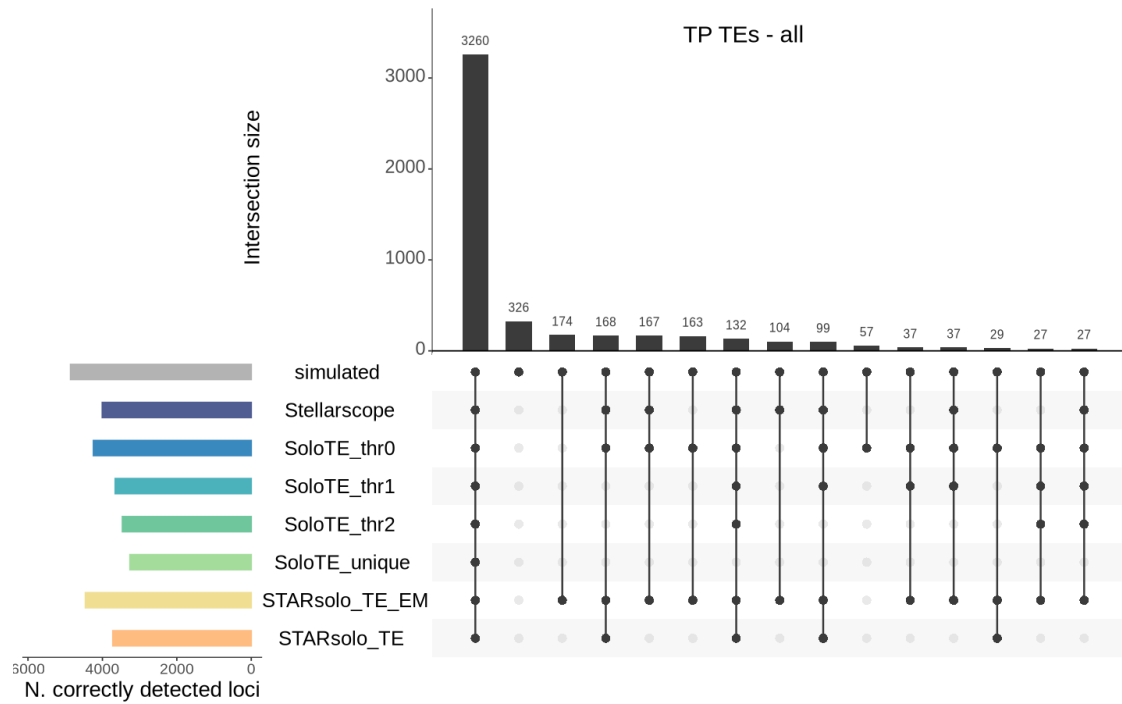
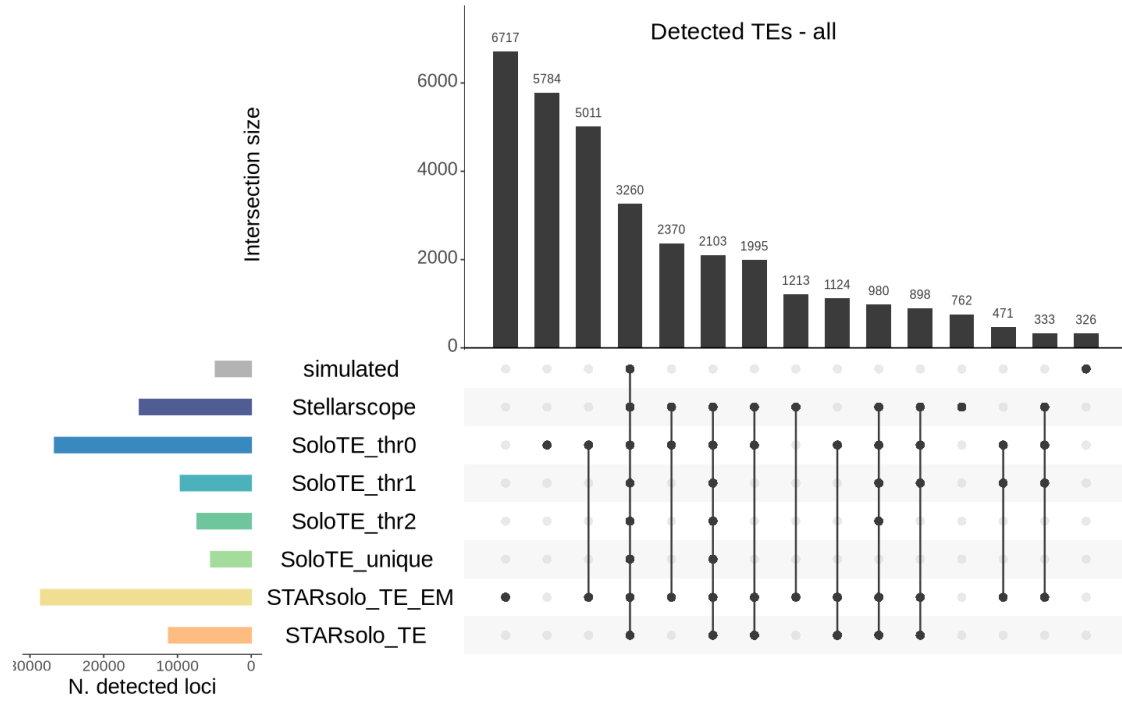
show(UpSetR::upset(fromList(FNlist), nintersects = 15,
  sets=names(colorTools), keep.order = T, sets.bar.color=colorTools,
  text.scale = c(2, 2, 2, 1.65, 2.2, 1.5),
  order.by=c("freq"),
  point.size=3, mb.ratio = c(0.5, 0.5),
  set_size.show=F, set_size.scale_max= max(lengths(FNlist))+1000,
  ↪#show.numbers = F,
  nsets=length(c(objList, splatter_obj)),
  sets.x.label="N. FN",
  mainbar.y.label="Intersection size")
)
grid.text(paste0("FN TEs - ", sample ),x = 0.65, y=0.95,
  ↪gp=gpar(fontsize=18))
  #/dev.off()
}

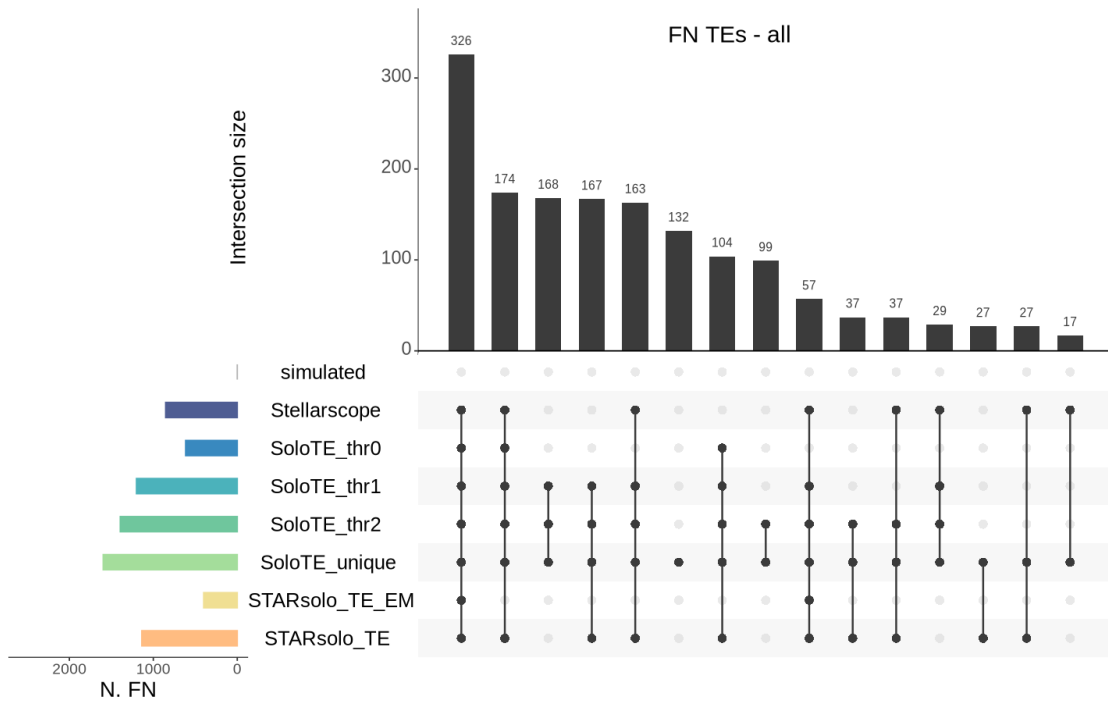
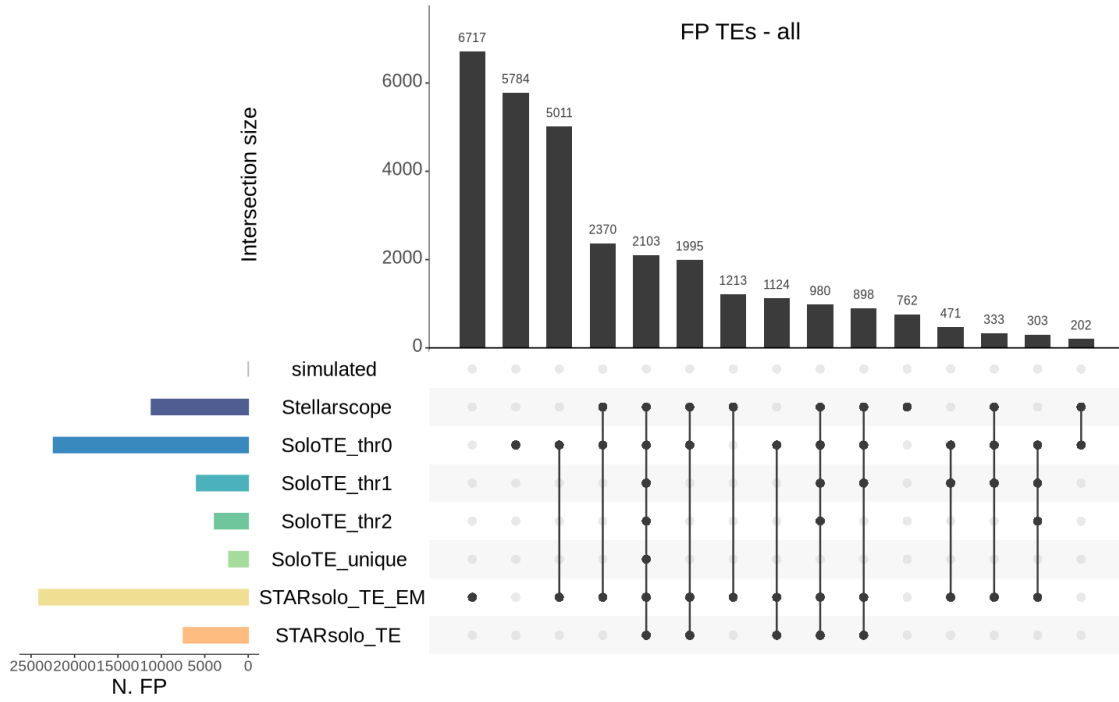
```

```

[1] "all"
[1] "STARsolo_TE"
[1] "STARsolo_TE_EM"
[1] "SoloTE_unique"
[1] "SoloTE_thr2"
[1] "SoloTE_thr1"
[1] "SoloTE_thr0"
[1] "Stellarscope"
[1] "simulated"

```





1.1.2 Upsets of detected Genes

```
[4]: options(repr.plot.width=12, repr.plot.height=7)

for (sample in samples){

  print(sample)

  TPgenelist <- list()
  FPgenelist <- list()
  FNgenelist <- list()
  detectedgeneList <- list()

  for (tool in c(names(objGeneList), "simulated")){

    print(tool)
    if (tool == "simulated") {
      obj <- splatter_objs_genes[[sample]]
    }else {
      obj <- objGeneList[[tool]][[sample]]
    }

    detectedgeneList[[sub("^(.*)_.*$", "\\1", obj@project.name)]] <-
    ↪Features(obj)
    TPgenelist[[sub("^(.*)_.*$", "\\1", obj@project.name)]] <-
    ↪intersect(Features(splatter_objs_genes[[sample]]), Features(obj))
    FNgenelist[[sub("^(.*)_.*$", "\\1", obj@project.name)]] <-
    ↪setdiff(Features(splatter_objs_genes[[sample]]), Features(obj))
    FPgenelist[[sub("^(.*)_.*$", "\\1", obj@project.name)]] <-
    ↪setdiff(Features(obj), Features(splatter_objs_genes[[sample]]))
  }

  options(repr.plot.width=12, repr.plot.height=6)
  # pdf(file=paste0("figures/figures", "_", dataset_id, "_", sample, "/"
  ↪upset_detected.pdf"), width = 12, height = 6)
  show(UpSetR::upset(fromList(detectedgeneList), nintersects = 15,
    sets=names(colorTools[c("simulated", names(objGeneList))]), keep.
  ↪order = T, sets.bar.color=colorTools[c("simulated", names(objGeneList))],
    text.scale = c(2, 2, 2, 1.65, 2.5, 1.5),
    order.by=c("freq"),
    point.size=3, mb.ratio = c(0.5, 0.5),
    set_size.show=F, set_size.scale_max=
  ↪max(lengths(detectedgeneList))+500, #show.numbers = F,
    nsets=length(objGeneList)+1,
    sets.x.label="N. detected genes",
    mainbar.y.label="Intersection size")
```

```

    )
    grid.text(paste0("Detected Genes - ", sample ),x = 0.65, y=0.95,
    ↪gp=gpar(fontsize=18))
    #dev.off()

    #pdf(file=paste0("figures/figures", "_", dataset_id, "_",sample,"/
    ↪upset_detected_TP.pdf"), width = 9, height = 6)
    show(UpSetR::upset(fromList(TPgenelist), nintersects = 15,
        sets=names(colorTools[c("simulated", names(objGeneList))]), keep.
    ↪order = T, sets.bar.color=colorTools[c("simulated", names(objGeneList))],
        text.scale = c(2, 2, 2, 1.65, 2.2, 1.5),
        order.by=c("freq"),
        point.size=3, mb.ratio = c(0.5, 0.5),
        set_size.show=F, set_size.scale_max= max(lengths(TPgenelist))+5000,
    ↪#show.numbers = F,
        nsets=length(c(objGeneList, splatter_obj)),
        sets.x.label="N. correctly detected Genes",
        mainbar.y.label="Intersection size")
    )
    grid.text(paste0("TP Genes - ", sample ),x = 0.65, y=0.95,
    ↪gp=gpar(fontsize=18))
    #dev.off()

    options(repr.plot.width=9, repr.plot.height=6)
    # pdf(file=paste0("figures/figures_", dataset_id, "_",sample,"/
    ↪upset_detected_TP.pdf"), width = 9, height = 6)
    show(UpSetR::upset(fromList(FPgenelist), nintersects = 15,
        sets=names(colorTools[c("simulated", names(objGeneList))]), keep.
    ↪order = T, sets.bar.color=colorTools[c("simulated", names(objGeneList))],
        text.scale = c(2, 2, 2, 1.65, 2.2, 1.5),
        order.by=c("freq"),
        point.size=3, mb.ratio = c(0.5, 0.5),
        set_size.show=F, set_size.scale_max= max(lengths(FPgenelist))+100,
    ↪#show.numbers = F,
        nsets=length(c(objGeneList, splatter_obj)),
        sets.x.label="N. FPGenes",
        mainbar.y.label="Intersection size")
    )
    # grid.text(paste0("FP Genes - ", sample ),x = 0.65, y=0.95,
    ↪gp=gpar(fontsize=18))
    # dev.off()
}

```

```

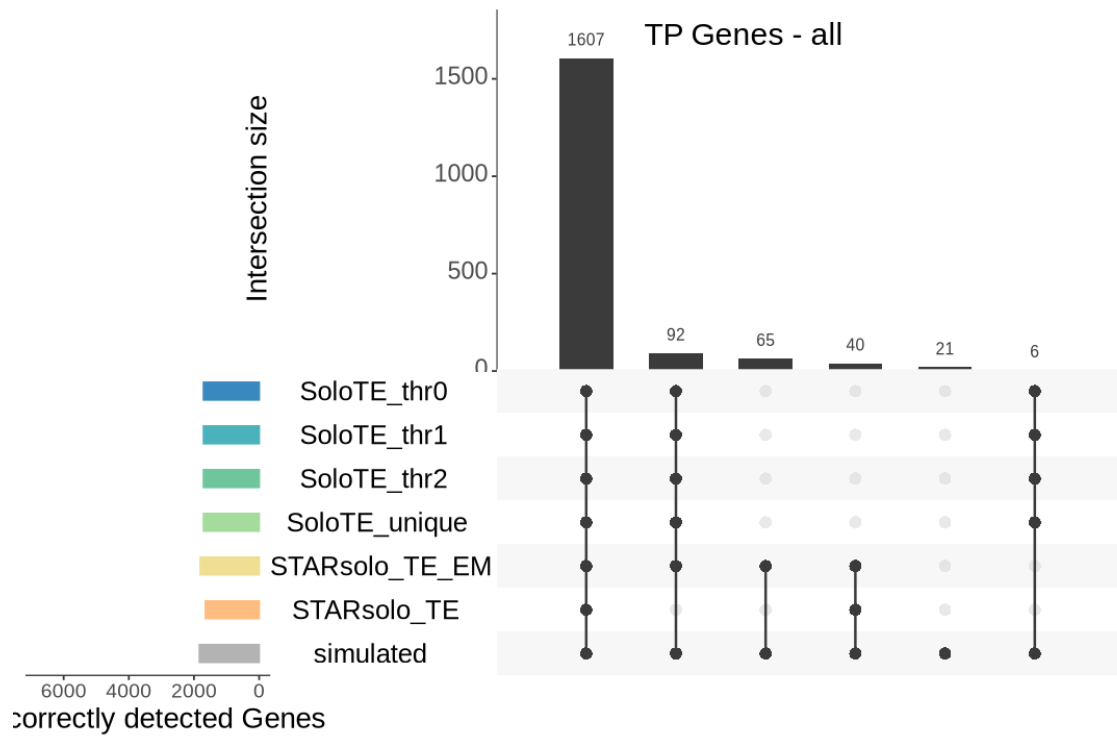
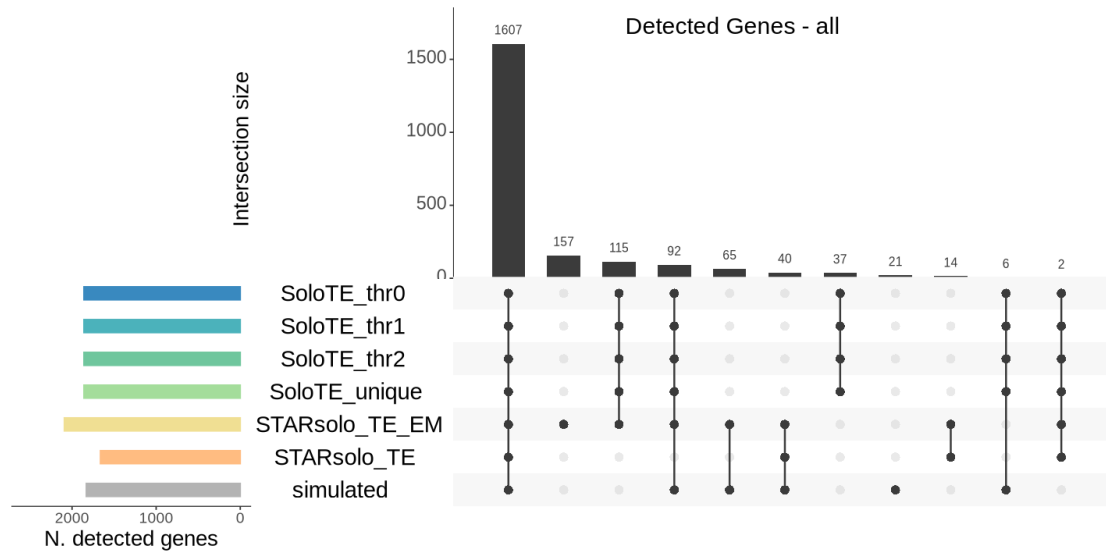
[1] "all"
[1] "STARsolo_TE"
[1] "STARsolo_TE_EM"
[1] "SoloTE_unique"

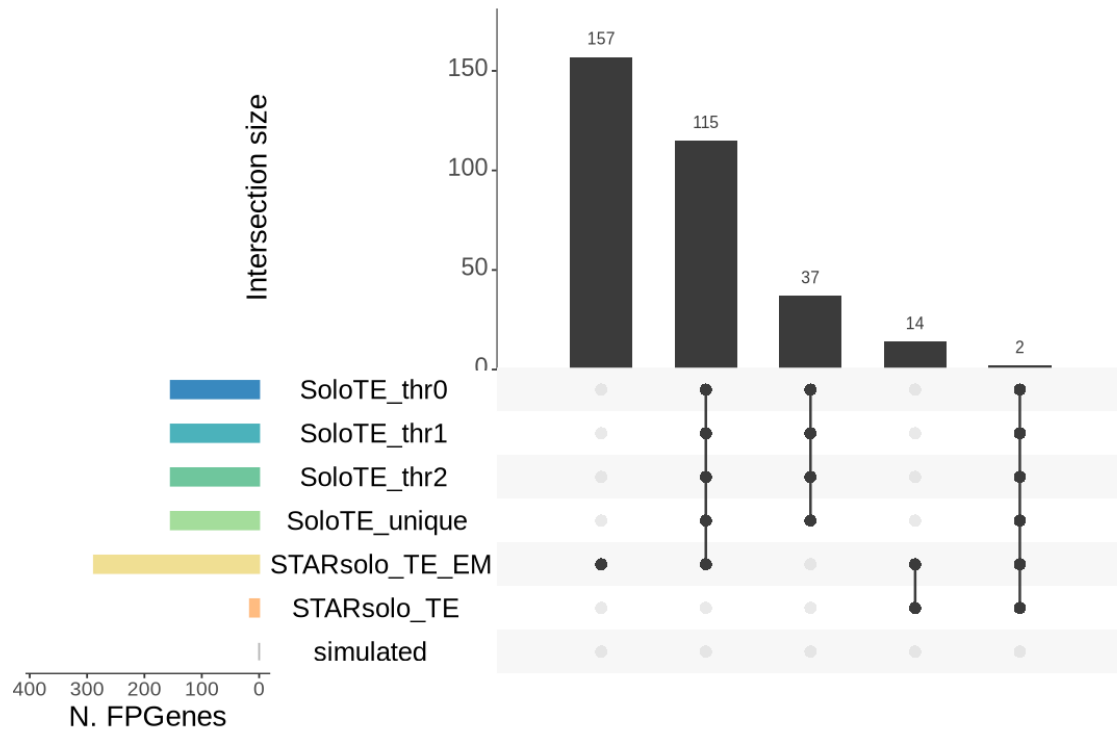
```

```

[1] "SoloTE_thr2"
[1] "SoloTE_thr1"
[1] "SoloTE_thr0"
[1] "simulated"

```





2 TE - gene intresection

```
[5]: library(rtracklayer)

gene_annotation <- rtracklayer::import("/mnt/transfer/TEbenchmarking/Rscripts/
↳ annotation/gencode.vM10.primary_assembly.annotation.gtf")
names(mcols(gene_annotation))

gene_annotation_df <- as.data.frame(gene_annotation)

# Extract only exon entries
exons <- gene_annotation[gene_annotation$type == "exon"]
names(exons) <- exons$gene_name

# Gene bodies (type == "gene")
gene_bodies <- gene_annotation[gene_annotation$type == "gene"]
names(gene_bodies) <- gene_bodies$gene_name
```

1. 'source' 2. 'type' 3. 'score' 4. 'phase' 5. 'gene_id' 6. 'gene_type' 7. 'gene_status'
8. 'gene_name' 9. 'level' 10. 'havana_gene' 11. 'transcript_id' 12. 'transcript_type'
13. 'transcript_status' 14. 'transcript_name' 15. 'transcript_support_level' 16. 'tag' 17. 'havana_transcript'
18. 'exon_number' 19. 'exon_id' 20. 'protein_id' 21. 'ccdsid' 22. 'ont'

2.1 Detected TEs overlapping exons

```
[10]: # =====  
# Genomic Classification of Transposable Elements (TEs)  
# =====  
# This script iterates through samples and tools (including the simulated data)  
# to map TE loci against exons. It identifies overlapping transcripts and  
# classifies each TE as exonic, intronic, or intergenic.  
  
# Initialize nested lists to store results per sample and tool  
overlapDfList_TE      <- list()  
classificationList_TE <- list()  
  
for (sample in samples) {  
  overlapDfList_TE[[sample]] <- list()  
  classificationList_TE[[sample]] <- list()  
  
  # Include "simulated" data as a pseudo-tool alongside actual tools  
  tools_and_sim <- c(names(objList), "simulated")  
  
  for (tool in tools_and_sim) {  
    print(paste("Processing:", sample, "-", tool))  
  
    ### 1. Extract TE Loci & Build GRanges -----  
    # Fetch the correct object depending on whether it's simulated or real  
    ↪data  
    if (tool == "simulated") {  
      obj <- splatter_objs[[sample]]  
    } else {  
      obj <- objList[[tool]][[sample]]  
    }  
  
    # Filter genomic coordinates from the conversion table for detected  
    ↪features  
    bed_TE_tool <- conversion_table[  
      conversion_table$stellarscopeID %in% Features(obj),  
      c("chr", "start", "end", "strand", "stellarscopeID")  
    ]  
    rownames(bed_TE_tool) <- bed_TE_tool$stellarscopeID  
  
    # Convert dataframe to a GRanges object for genomic overlap analysis  
    gr_TE_tool <- makeGRangesFromDataFrame(bed_TE_tool)  
    names(gr_TE_tool) <- rownames(bed_TE_tool)  
  
    ### 2. Calculate Exon Overlaps -----  
    # Find overlaps between TE features and exon annotations
```



```

exonOverlaps <- findOverlaps(gr_TE_tool, exons, minoverlap = 1)

# Compute exact overlap length in base pairs
overlap_bp <- width(pintersect(
  gr_TE_tool[queryHits(exonOverlaps)],
  exons[subjectHits(exonOverlaps)]
))

# Build overlap dataframe and flag features present in the simulation
overlap_df <- data.frame(
  TE = names(gr_TE_tool)[queryHits(exonOverlaps)],
  exon = names(exons)[subjectHits(exonOverlaps)],
  gene_id = mcols(exons)$gene_id[subjectHits(exonOverlaps)],
  transcript_id =   

↳ mcols(exons)$transcript_id[subjectHits(exonOverlaps)],
  gene_name = mcols(exons)$gene_name[subjectHits(exonOverlaps)],
  overlap_bp = overlap_bp
) %>%
  dplyr::mutate(
    TEinSimulation = TE %in% Features(splatter_objs[[sample]]),
    geneInSimulation = gene_name %in%   

↳ Features(splatter_objs_genes[[sample]])
  ) %>%
  # Resolve multi-mapping TEs by keeping only the highest-overlapping   

↳ exon
  dplyr::group_by(TE) %>%
  dplyr::slice_max(overlap_bp, n = 1, with_ties = FALSE) %>%
  dplyr::ungroup()

### 3. Hierarchical Genomic Classification -----
# Exonic: Overlaps at least 1bp of an exon
exonic_TEs <- unique(overlap_df$TE)

# Genic: Overlaps any part of the broader gene body
genic_TEs <- unique(names(gr_TE_tool)[queryHits(
  findOverlaps(gr_TE_tool, gene_bodies, minoverlap = 1)
)])

# Intronic: Within gene body but completely misses exons
intronic_TEs <- setdiff(genic_TEs, exonic_TEs)
all_TEs <- names(gr_TE_tool)

# Intergenic: Falls entirely outside known gene bodies
intergenic_TEs <- setdiff(all_TEs, genic_TEs)

```

```

### 4. Construct Final Classification Table -----
classification_df <- data.frame(
  TE      = all_TEs,
  category = dplyr::case_when(
    all_TEs %in% exonic_TEs ~ "exonic",
    all_TEs %in% intronic_TEs ~ "intronic",
    TRUE ~ "intergenic"
  )
) %>%
  # Append exon-specific metadata to TEs classified as exonic
  ↪(overlap_length, whether the gene is simulated)
  dplyr::left_join(
    dplyr::select(overlap_df, TE, gene_id, gene_name,
                  transcript_id, overlap_bp, geneInSimulation),
    by = "TE"
  ) %>%
  dplyr::mutate(
    TEinSimulation = TE %in% Features(splatter_objs[[sample]])
  )

### 5. Store Output
  ↪ -----
  overlapDfList_TE[[sample]][[tool]] <- overlap_df
  classificationList_TE[[sample]][[tool]] <- classification_df
}
}

```

```
[1] "Processing: all - STARsolo_TE"
```

Warning message in .merge_two_Seqinfo_objects(x, y):

"Each of the 2 combined objects has sequence levels not in the other:

- in 'x': GL456382.1

- in 'y': chrM, GL456216.1, GL456219.1, GL456221.1, GL456239.1, GL456350.1,
GL456381.1, JH584292.1, JH584293.1, JH584294.1, JH584295.1, JH584296.1,
JH584298.1, JH584299.1, JH584303.1

Make sure to always combine/compare objects based on the same reference
genome (use suppressWarnings() to suppress this warning)."

Warning message in .merge_two_Seqinfo_objects(x, y):

"Each of the 2 combined objects has sequence levels not in the other:

- in 'x': GL456382.1

- in 'y': chrM, GL456216.1, GL456219.1, GL456221.1, GL456239.1, GL456350.1,
GL456381.1, JH584292.1, JH584293.1, JH584294.1, JH584295.1, JH584296.1,
JH584298.1, JH584299.1, JH584303.1

Make sure to always combine/compare objects based on the same reference
genome (use suppressWarnings() to suppress this warning)."

Warning message in .merge_two_Seqinfo_objects(x, y):

"Each of the 2 combined objects has sequence levels not in the other:

```

- in 'x': GL456382.1
- in 'y': chrM, GL456216.1, GL456219.1, GL456221.1, GL456239.1, GL456350.1,
GL456381.1, JH584292.1, JH584293.1, JH584294.1, JH584295.1, JH584296.1,
JH584298.1, JH584299.1, JH584303.1
Make sure to always combine/compare objects based on the same reference
genome (use suppressWarnings() to suppress this warning)."
```

[1] "Processing: all - STARsolo_TE_EM"

```
Warning message in .merge_two_Seqinfo_objects(x, y):
"Each of the 2 combined objects has sequence levels not in the other:
- in 'x': GL456382.1, JH584300.1, JH584301.1
- in 'y': chrM, GL456216.1, GL456239.1, GL456381.1, JH584292.1, JH584294.1,
JH584295.1
Make sure to always combine/compare objects based on the same reference
genome (use suppressWarnings() to suppress this warning)."
```

```
Warning message in .merge_two_Seqinfo_objects(x, y):
"Each of the 2 combined objects has sequence levels not in the other:
- in 'x': GL456382.1, JH584300.1, JH584301.1
- in 'y': chrM, GL456216.1, GL456239.1, GL456381.1, JH584292.1, JH584294.1,
JH584295.1
Make sure to always combine/compare objects based on the same reference
genome (use suppressWarnings() to suppress this warning)."
```

```
Warning message in .merge_two_Seqinfo_objects(x, y):
"Each of the 2 combined objects has sequence levels not in the other:
- in 'x': GL456382.1, JH584300.1, JH584301.1
- in 'y': chrM, GL456216.1, GL456239.1, GL456381.1, JH584292.1, JH584294.1,
JH584295.1
Make sure to always combine/compare objects based on the same reference
genome (use suppressWarnings() to suppress this warning)."
```

[1] "Processing: all - SoloTE_unique"

[1] "Processing: all - SoloTE_thr2"

[1] "Processing: all - SoloTE_thr1"

[1] "Processing: all - SoloTE_thr0"

[1] "Processing: all - Stellarscope"

```
Warning message in .merge_two_Seqinfo_objects(x, y):
"Each of the 2 combined objects has sequence levels not in the other:
- in 'x': GL456382.1
- in 'y': chrM, GL456216.1, GL456219.1, GL456239.1, GL456350.1, GL456381.1,
JH584292.1, JH584293.1, JH584294.1, JH584295.1, JH584296.1, JH584298.1,
JH584299.1, JH584303.1
Make sure to always combine/compare objects based on the same reference
genome (use suppressWarnings() to suppress this warning)."
```

```
Warning message in .merge_two_Seqinfo_objects(x, y):
"Each of the 2 combined objects has sequence levels not in the other:
- in 'x': GL456382.1
- in 'y': chrM, GL456216.1, GL456219.1, GL456239.1, GL456350.1, GL456381.1,
JH584292.1, JH584293.1, JH584294.1, JH584295.1, JH584296.1, JH584298.1,
```

```
JH584299.1, JH584303.1
  Make sure to always combine/compare objects based on the same reference
  genome (use suppressWarnings() to suppress this warning)."
```

Warning message in `.merge_two_Seqinfo_objects(x, y)`:

```
"Each of the 2 combined objects has sequence levels not in the other:
- in 'x': GL456382.1
- in 'y': chrM, GL456216.1, GL456219.1, GL456239.1, GL456350.1, GL456381.1,
JH584292.1, JH584293.1, JH584294.1, JH584295.1, JH584296.1, JH584298.1,
JH584299.1, JH584303.1
  Make sure to always combine/compare objects based on the same reference
  genome (use suppressWarnings() to suppress this warning)."
```

```
[1] "Processing: all - simulated"
```

```
[12]: head(overlapDfList_TE[[sample]][[tool]])
      head(classificationList_TE[[sample]][[tool]])
```

	TE <chr>	exon <chr>	gene_id <chr>	transcript_id <chr>
A tibble: 6 × 8	AmnSINE1-dup1253	Malt1	ENSMUSG00000032688.7	ENSMUST00000049248.5
	AmnSINE1-dup163	Gm25966	ENSMUSG00000087794.1	ENSMUST00000157169.1
	AmnSINE1-dup389	Gm26486	ENSMUSG00000088646.1	ENSMUST00000158021.1
	AmnSINE1-dup576	Mob2	ENSMUSG00000025147.7	ENSMUST00000211633.1
	Arthur1-dup4	4921511E07Rik	ENSMUSG00000104133.1	ENSMUST00000194790.1
	B1-Mus1-dup44044	Gm17545	ENSMUSG00000091159.1	ENSMUST00000168332.1

	TE <chr>	category <chr>	gene_id <chr>	gene_name <chr>	transcript_id <chr>	overlap_bp <int>	gene_id <chr>
A data.frame: 6 × 8	1 X3-LINE	intronic	NA	NA	NA	NA	NA
	2 IAPez-int-dup9	intergenic	NA	NA	NA	NA	NA
	3 RLTR41-dup1	intergenic	NA	NA	NA	NA	NA
	4 MT2-Mm-dup5	intergenic	NA	NA	NA	NA	NA
	5 Looper-dup1	intergenic	NA	NA	NA	NA	NA
	6 B1-Mus1-dup96	intergenic	NA	NA	NA	NA	NA

3 Checkpoint

```
[1]: #save.image("workspaces/mm_RA_wGenes_afterClassification.Rdata")
      load("workspaces/mm_RA_wGenes_afterClassification.Rdata")
```

```
[2]: gc()
```

		used (Mb)	gc trigger (Mb)	max used (Mb)
A matrix: 2 × 6 of type dbl	Ncells	21000952	1121.6	39486914
	Vcells	485192706	3701.8	662749398

```
[8]: # Flatten classificationList_TE into one df
      classification_all <- purrr::imap_dfr(classificationList_TE, ~
      ↪function(sample_list, sample){
```

```

    purrr::imap_dfr(sample_list, function(df, tool){
      df %>% dplyr::mutate(sample = sample, tool = tool)
    })
  })

options(repr.plot.width=8, repr.plot.height=7)

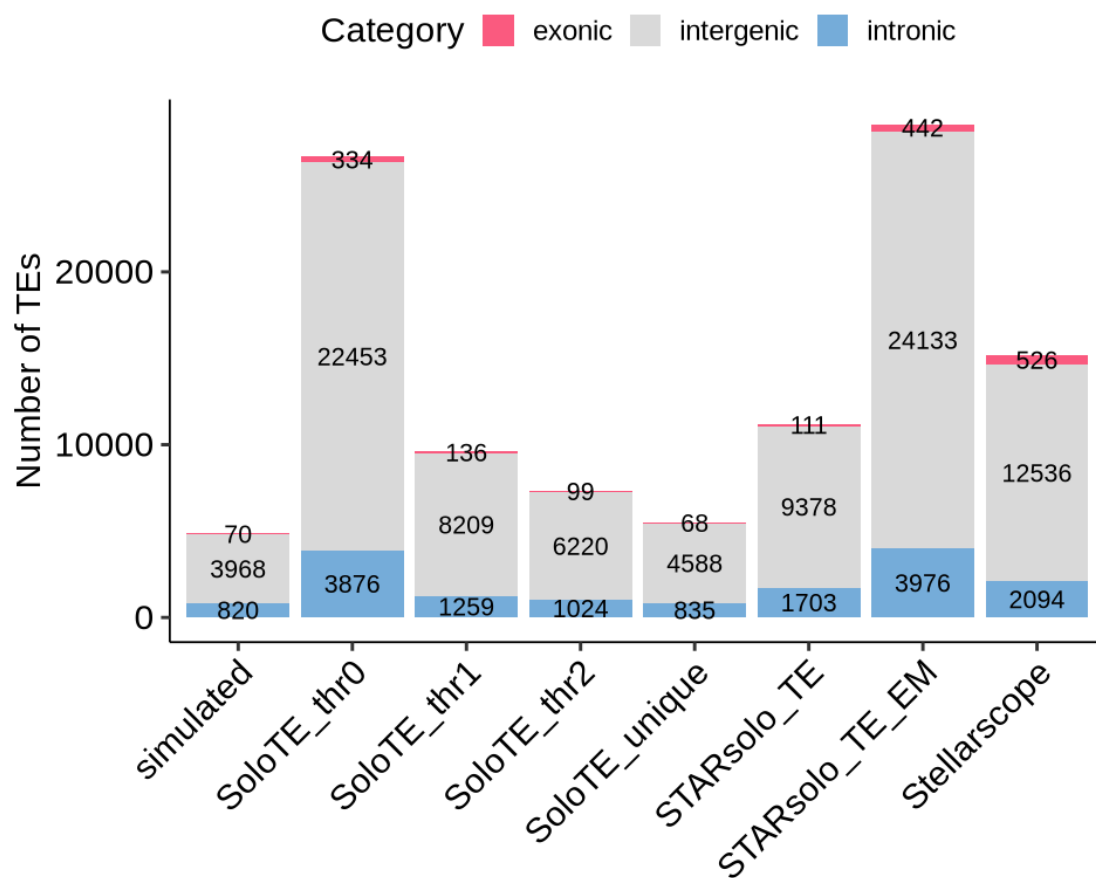
# Define a base text size for all plots
text_size <- 18

# Category counts per tool/sample (stacked bar)
classification_all %>%
  dplyr::count(sample, tool, category) %>%
  ggplot(aes(x = tool, y = n, fill = category)) +
  geom_col(position = "stack") +
  geom_text(aes(label = n), position = position_stack(vjust = 0.5), size = 4.
↪5) + # Larger text
  scale_fill_manual(values = c(exonic = "#fa5a7f", intronic = "#75acd9", ↪
↪intergenic = "grey85")) +
  labs(title = "TE genomic category distribution",
       x = NULL, y = "Number of TEs", fill = "Category") +
  theme_pubr(base_size = text_size) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
ggsave(paste0("figures/figures_", dataset_id, "_", sample, "/"
↪TEgenomicCategory_numbers.pdf"), device="pdf", width=8, height=7)

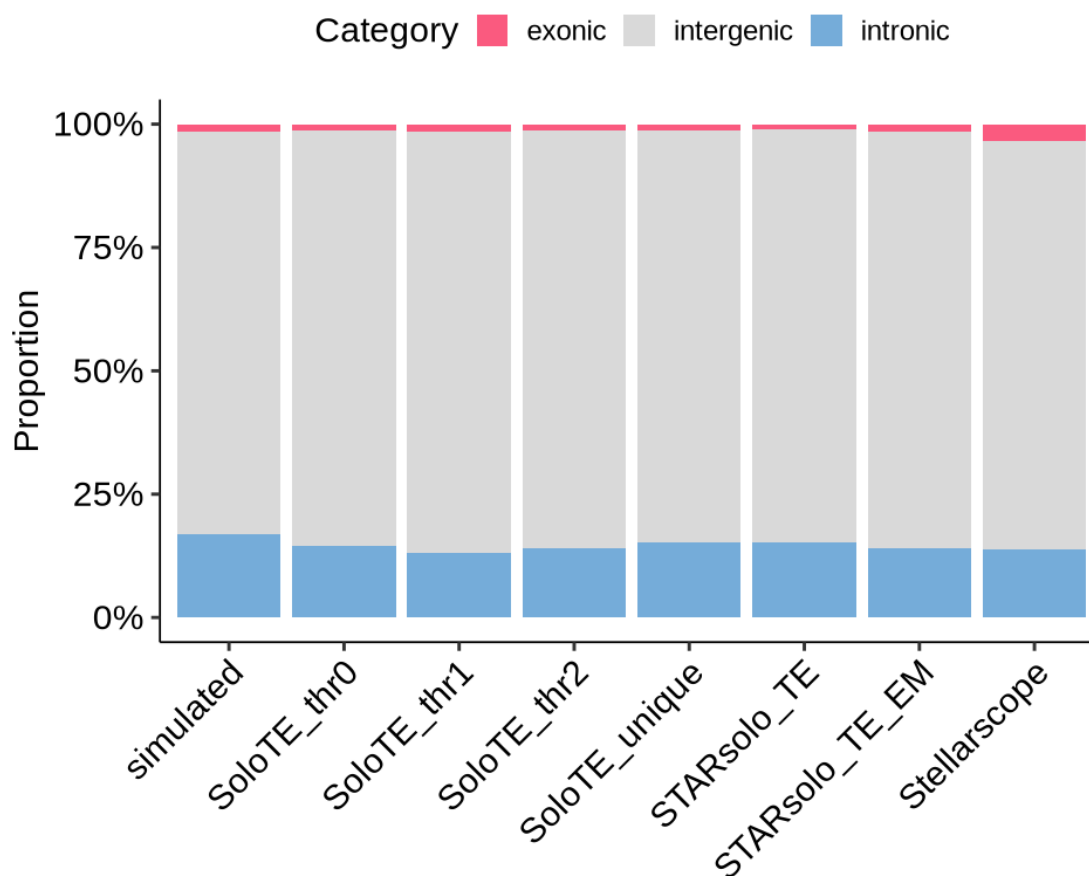
# Category proportions per tool/sample (filled bar)
classification_all %>%
  dplyr::count(sample, tool, category) %>%
  ggplot(aes(x = tool, y = n, fill = category)) +
  geom_col(position = "fill") +
  scale_y_continuous(labels = scales::percent) +
  scale_fill_manual(values = c(exonic = "#fa5a7f", intronic = "#75acd9", ↪
↪intergenic = "grey85")) +
  labs(title = "TE genomic category proportions",
       x = NULL, y = "Proportion", fill = "Category") +
  theme_pubr(base_size = text_size) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
ggsave(paste0("figures/figures_", dataset_id, "_", sample, "/"
↪TEgenomicCategory_percentages.pdf"), device="pdf", width=8, height=7)

```

TE genomic category distribution



TE genomic category proportions



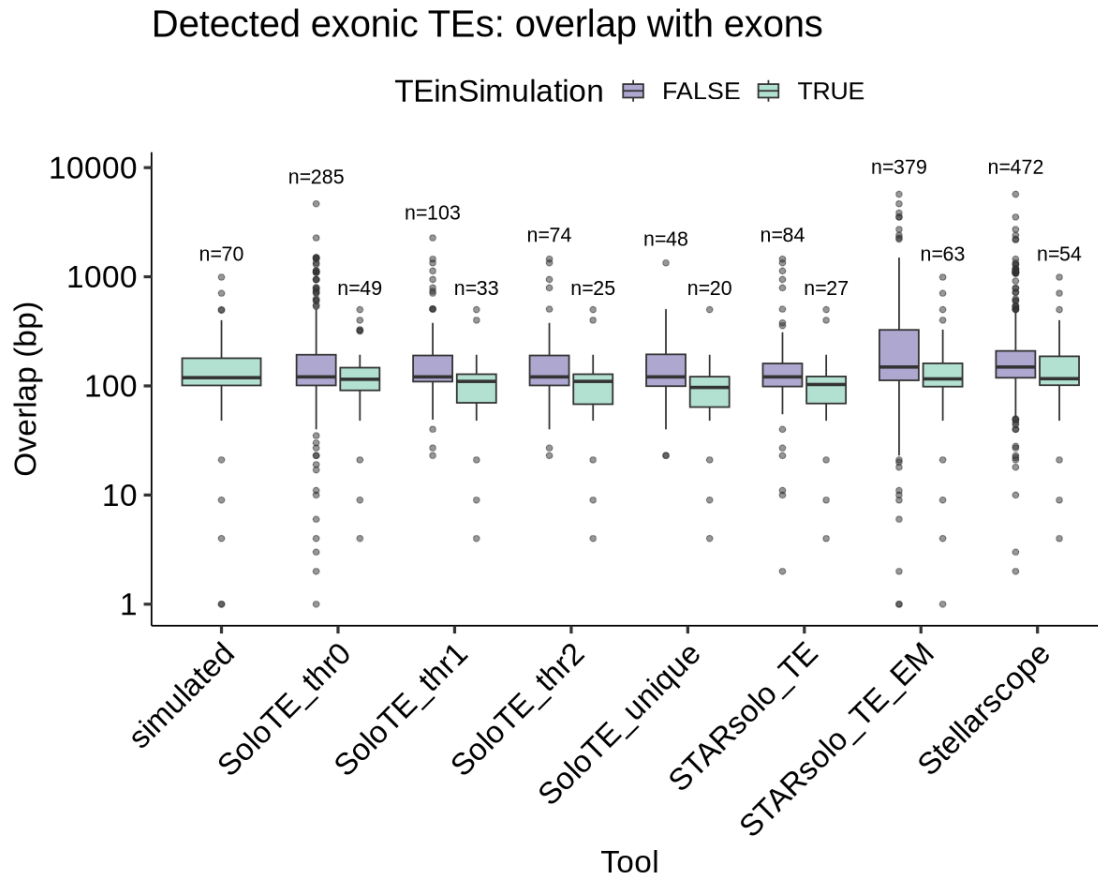
```
[11]: options(repr.plot.width=10, repr.plot.height=8)
text_size <- 20

classification_all %>%
  dplyr::filter(category == "exonic") %>%
  ggplot(aes(x = tool, y = overlap_bp, fill = TEinSimulation)) +
  geom_boxplot(alpha = 0.5) +
  stat_summary(
    fun.data = function(y) {
      data.frame(
        y = max(y, na.rm = TRUE) * 1.05, # slightly above max
        label = paste0("n=", length(y))
      )
    },
    geom = "text",
    size = 4.5,
```

```

position = position_dodge(0.75),
vjust = 0
) +
scale_y_continuous(transform = "log10") +
scale_fill_manual(values = c("TRUE" = "#67c1a3", "FALSE" = "#5e50a1")) +
labs(
  title = "Detected exonic TEs: overlap with exons",
  x = "Tool",
  y = "Overlap (bp)"
) +
theme_pubr(base_size = text_size) +
theme(axis.text.x = element_text(angle = 45, hjust = 1))
ggsave(paste0("figures/figures_", dataset_id, "_", sample, "/"
  ↪TEgene_overlap_FPandTP.pdf"), device="pdf", width=10, height=8)

```



```

[20]: # plot number of genes detected by SoloTE in scenario 1 and 2
options(repr.plot.width=5, repr.plot.height=5)

detectedGenesSoloTE <- list("Scenario1-old" = 22,

```



```

        "Scenario1-young" = 208,
        "Scenario2-mixed" = 154)

df <- data.frame(
  scenario = factor(names(detectedGenesSoloTE),
                    levels = names(detectedGenesSoloTE),
                    labels = c("Scenario 1\n(old TEs)",
                              "Scenario 1\n(young TEs)",
                              "Scenario 2\n(mixed)")),
  n_genes = unlist(detectedGenesSoloTE)
)

ggplot(df, aes(x = scenario, y = n_genes, fill = scenario)) +
  geom_col(width = 0.6) +
  geom_text(aes(label = n_genes), vjust = -0.6, fontface = "bold", size = 6) +
  scale_fill_manual(values = c("#A58065", "#7FCFF2", "#92A8AC")) +
  labs(
    title = "SoloTE: genes falsely called as expressed\nfrom TE-derived reads,
↳alone",
    x = NULL,
    y = "Falsely detected genes (SoloTE)"
  ) +
  ylim(0, max(df$n_genes) * 1.05) +
  theme_pubr() +
  theme(
    legend.position = "none",
    plot.title = element_text(hjust = 0.5, size = 16),
    text = element_text(size = 16)
  )

ggsave(paste0("figures/figures_", dataset_id, "_", sample, "/"
↳SoloTE_incorrectlyDetectedGenes_scenario1and2.pdf"), device="pdf", width=5,
↳height=5)

```



4 Gene intersection plots

```
[64]: # compute intersection between list of loci of each tool and gene annotation
for (sample in samples){

  for(tool in names(objList)){

    print(tool)
    overlapDfList_TE[[sample]][[tool]]$detection <- NA

    ␣
    ↪overlapDfList_TE[[sample]][[tool]]$detection[overlapDfList_TE[[sample]][[tool]]$TE_
    ↪in% TPlist[[tool]]] <- "TPs"

    ␣
    ↪overlapDfList_TE[[sample]][[tool]]$detection[overlapDfList_TE[[sample]][[tool]]$TE_
    ↪in% FPlist[[tool]]] <- "FPs"
```

```
}
}
```

```
[1] "STARsolo_TE"
[1] "STARsolo_TE_EM"
[1] "SoloTE_unique"
[1] "SoloTE_thr2"
[1] "SoloTE_thr1"
[1] "SoloTE_thr0"
[1] "Stellarscope"
```

```
[ ]: percPosOverlapGenesDf <- NULL

for (sample in samples){

  for(tool in names(objList)){

    print(tool)

    df <- cbind(melt(table(overlapDfList_TE[[sample]][[tool]]$detection,
    overlapDfList_TE[[sample]][[tool]]$geneInSimulation )), tool)
    colnames(df) <- c("Detection", "ExpressedGene", "Number", "Tool")
    print(df)
    percPosOverlapGenesDf <- rbind(percPosOverlapGenesDf, df)

  }

}
```

```
[1] "STARsolo_TE"
  Detection ExpressedGene Number      Tool
1      FPs          FALSE     59 STARsolo_TE
2      TPs          FALSE     23 STARsolo_TE
3      FPs           TRUE     25 STARsolo_TE
4      TPs           TRUE      4 STARsolo_TE
[1] "STARsolo_TE_EM"
  Detection ExpressedGene Number      Tool
1      FPs          FALSE    225 STARsolo_TE_EM
2      TPs          FALSE     47 STARsolo_TE_EM
3      FPs           TRUE    154 STARsolo_TE_EM
4      TPs           TRUE     16 STARsolo_TE_EM
[1] "SoloTE_unique"
  Detection ExpressedGene Number      Tool
1      FPs          FALSE     27 SoloTE_unique
2      TPs          FALSE     19 SoloTE_unique
3      FPs           TRUE     21 SoloTE_unique
4      TPs           TRUE      1 SoloTE_unique
[1] "SoloTE_thr2"
```

	Detection	ExpressedGene	Number	Tool
1	FPs	FALSE	49	SoloTE_thr2
2	TPs	FALSE	23	SoloTE_thr2
3	FPs	TRUE	25	SoloTE_thr2
4	TPs	TRUE	2	SoloTE_thr2

[1] "SoloTE_thr1"

	Detection	ExpressedGene	Number	Tool
1	FPs	FALSE	63	SoloTE_thr1
2	TPs	FALSE	25	SoloTE_thr1
3	FPs	TRUE	40	SoloTE_thr1
4	TPs	TRUE	8	SoloTE_thr1

[1] "SoloTE_thr0"

	Detection	ExpressedGene	Number	Tool
1	FPs	FALSE	181	SoloTE_thr0
2	TPs	FALSE	34	SoloTE_thr0
3	FPs	TRUE	104	SoloTE_thr0
4	TPs	TRUE	15	SoloTE_thr0

[1] "Stellarscope"

	Detection	ExpressedGene	Number	Tool
1	FPs	FALSE	102	Stellarscope
2	TPs	FALSE	44	Stellarscope
3	FPs	TRUE	370	Stellarscope
4	TPs	TRUE	10	Stellarscope

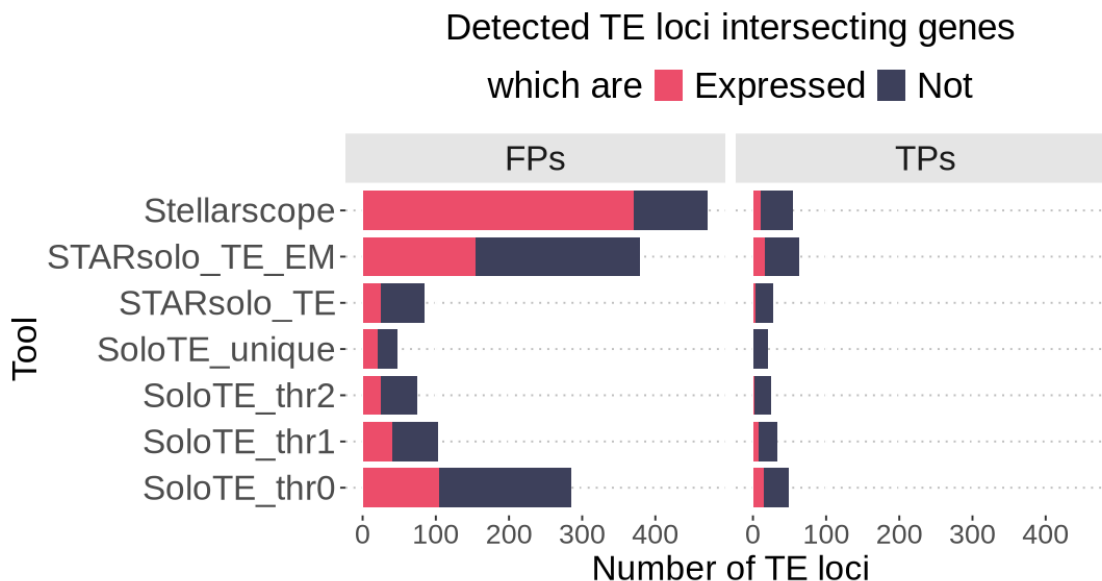
```
[66]: percPosOverlapGenesDf$ExpressedGene <- as.
      ↪character(percPosOverlapGenesDf$ExpressedGene)
```

```
[ ]: options(repr.plot.width=9, repr.plot.height=4.75)

ggplot(percPosOverlapGenesDf, aes(y=Tool, x=Number, fill=ExpressedGene)) +
  facet_wrap(~Detection) +
  geom_col(width = 0.85) +
  scale_fill_manual(values=c("TRUE"="#EC4D6A", "FALSE"="#3d405b"),
                    labels = c("TRUE"="Expressed", "FALSE"="Not"),
                    breaks=c("TRUE", "FALSE"), name="which are") +
  theme_pubclean() +
  ggtitle("Detected TE loci intersecting genes") +
  xlab("Number of TE loci") +
  theme(text=element_text(size=20),
        axis.text.y = element_text(size=20),
        axis.label = element_text(size=18),
        legend.title = element_text(size=21),
        legend.text = element_text(size=21),
        strip.text = element_text(size = 20),
        strip.background = element_rect(fill="grey90"),
        plot.title = element_text(size=21, hjust=0.5),
        plot.subtitle = element_text(size=20, hjust=0.5))
```

```
ggsave(paste0("figures/figures_", dataset_id, "_", sample, "/"
  ↳detectedTEsIntersectingGenes.pdf"),
  device="pdf", width=9, height=4.5)
```

Warning message in plot_theme(plot):
 "The `axis.label` theme element is not defined in the element hierarchy."



4.1 Detected Genes overlapping TEs

```
[68]: gc()

bed_TE <- conversion_table[,c("chr", "start", "end", "strand", "stellarscopeID")]
#rownames(bed_TE) <- bed_TE$stellarscopeID

gr_TE <- makeGRangesFromDataFrame(bed_TE, keep.extra.columns = TRUE )
```

		used	(Mb)	gc trigger	(Mb)	max used	(Mb)
A matrix: 2 × 6 of type dbl	Ncells	31429163	1678.5	55645654	2971.8	55645654	2971.8
	Vcells	498574866	3803.9	877976232	6698.5	841749691	6422.1

```
[ ]: overlapDfList_GENE <- list()
overlapDfList_GENE_unique <- list()

# compute intersection between list of loci of each tool and gene annotation
for (sample in samples){
  overlapDfList_GENE[[sample]] <- list()
```

```

for(tool in names(objGeneList)){

  print(tool)
  obj <- objGeneList[[tool]][[sample]]

  # get detected TE gr
  gr_GENE_tool <- exons[exons$gene_name %in% Features(obj)]

  exonOverlaps <- findOverlaps(gr_GENE_tool, gr_TE, minoverlap = 1)

  queryHits <- queryHits(exonOverlaps)
  subjectHits <- subjectHits(exonOverlaps)

  overlap_df <- data.frame(
    Exon = names(gr_GENE_tool)[queryHits],
    gene_id = mcols(gr_GENE_tool)$gene_id[queryHits],
    gene_name = mcols(gr_GENE_tool)$gene_name[queryHits],
    TE = mcols(gr_TE)$stellarscopeID[subjectHits]
  )

  # column indicating whether the overlapping TE is in the simulated TE
  ↪matrix
  overlap_df$TEInSimulation <- overlap_df$TE %in%
  ↪Features(splatter_objs[[sample]])

  overlapDfList_GENE[[sample]][[tool]] <- overlap_df

  # create df with only the unique lines, prioritizing TEs that are in
  ↪the simulation
  overlapDfList_GENE_unique[[sample]][[tool]] <-
  ↪overlapDfList_GENE[[sample]][[tool]] %>% distinct()
  overlapDfList_GENE_unique[[sample]][[tool]] <-
  ↪overlapDfList_GENE_unique[[sample]][[tool]] %>%
    group_by(gene_name) %>%
    slice_max(order_by = TEInSimulation, n = 1, with_ties = FALSE)
  ↪%>%
    ungroup()

}
}

```

```
[1] "STARsolo_TE"
```

Warning message in .merge_two_Seqinfo_objects(x, y):

"Each of the 2 combined objects has sequence levels not in the other:

- in 'x': chrM, JH584295.1

- in 'y': GL456213.1, GL456359.1, GL456360.1, GL456366.1, GL456367.1,
GL456368.1, GL456370.1, GL456378.1, GL456379.1, GL456382.1, GL456383.1,

```
GL456387.1, GL456389.1, GL456390.1, GL456392.1, GL456393.1, GL456394.1,  
GL456396.1, JH584300.1, JH584301.1, JH584302.1
```

```
Make sure to always combine/compare objects based on the same reference  
genome (use suppressWarnings() to suppress this warning)."
```

```
[1] "STARsolo_TE_EM"
```

```
Warning message in .merge_two_Seqinfo_objects(x, y):
```

```
"Each of the 2 combined objects has sequence levels not in the other:
```

```
- in 'x': chrM, JH584295.1  
- in 'y': GL456213.1, GL456359.1, GL456360.1, GL456366.1, GL456367.1,  
GL456368.1, GL456370.1, GL456378.1, GL456379.1, GL456382.1, GL456383.1,  
GL456387.1, GL456389.1, GL456390.1, GL456392.1, GL456393.1, GL456394.1,  
GL456396.1, JH584300.1, JH584301.1, JH584302.1
```

```
Make sure to always combine/compare objects based on the same reference  
genome (use suppressWarnings() to suppress this warning)."
```

```
[1] "SoloTE_unique"
```

```
Warning message in .merge_two_Seqinfo_objects(x, y):
```

```
"Each of the 2 combined objects has sequence levels not in the other:
```

```
- in 'x': chrM, JH584295.1  
- in 'y': GL456213.1, GL456359.1, GL456360.1, GL456366.1, GL456367.1,  
GL456368.1, GL456370.1, GL456378.1, GL456379.1, GL456382.1, GL456383.1,  
GL456387.1, GL456389.1, GL456390.1, GL456392.1, GL456393.1, GL456394.1,  
GL456396.1, JH584300.1, JH584301.1, JH584302.1
```

```
Make sure to always combine/compare objects based on the same reference  
genome (use suppressWarnings() to suppress this warning)."
```

```
[1] "SoloTE_thr2"
```

```
Warning message in .merge_two_Seqinfo_objects(x, y):
```

```
"Each of the 2 combined objects has sequence levels not in the other:
```

```
- in 'x': chrM, JH584295.1  
- in 'y': GL456213.1, GL456359.1, GL456360.1, GL456366.1, GL456367.1,  
GL456368.1, GL456370.1, GL456378.1, GL456379.1, GL456382.1, GL456383.1,  
GL456387.1, GL456389.1, GL456390.1, GL456392.1, GL456393.1, GL456394.1,  
GL456396.1, JH584300.1, JH584301.1, JH584302.1
```

```
Make sure to always combine/compare objects based on the same reference  
genome (use suppressWarnings() to suppress this warning)."
```

```
[1] "SoloTE_thr1"
```

```
Warning message in .merge_two_Seqinfo_objects(x, y):
```

```
"Each of the 2 combined objects has sequence levels not in the other:
```

```
- in 'x': chrM, JH584295.1  
- in 'y': GL456213.1, GL456359.1, GL456360.1, GL456366.1, GL456367.1,  
GL456368.1, GL456370.1, GL456378.1, GL456379.1, GL456382.1, GL456383.1,  
GL456387.1, GL456389.1, GL456390.1, GL456392.1, GL456393.1, GL456394.1,  
GL456396.1, JH584300.1, JH584301.1, JH584302.1
```

```
Make sure to always combine/compare objects based on the same reference  
genome (use suppressWarnings() to suppress this warning)."
```

```
[1] "SoloTE_thr0"
```

Warning message in `.merge_two_Seqinfo_objects(x, y)`:

"Each of the 2 combined objects has sequence levels not in the other:

- in 'x': chrM, JH584295.1

- in 'y': GL456213.1, GL456359.1, GL456360.1, GL456366.1, GL456367.1, GL456368.1, GL456370.1, GL456378.1, GL456379.1, GL456382.1, GL456383.1, GL456387.1, GL456389.1, GL456390.1, GL456392.1, GL456393.1, GL456394.1, GL456396.1, JH584300.1, JH584301.1, JH584302.1

Make sure to always combine/compare objects based on the same reference genome (use `suppressWarnings()` to suppress this warning)."

```
[70]: ## Detected genes overlapping TE loci
# compute intersection between list of loci of each tool and gene annotation
for (sample in samples){
```

```
  for(tool in names(objGeneList)){
```

```
    print(tool)
```

```
    overlapDfList_GENE_unique[[sample]][[tool]]$detection <- NA
```

```
    ␣
```

```
    ↪overlapDfList_GENE_unique[[sample]][[tool]]$detection[overlapDfList_GENE_unique[[sample]][[
```

```
    ↪%in% TPgenelist[[tool]]] <- "TPs"
```

```
    ␣
```

```
    ↪overlapDfList_GENE_unique[[sample]][[tool]]$detection[overlapDfList_GENE_unique[[sample]][[
```

```
    ↪%in% FPgenelist[[tool]]] <- "FPs"
```

```
  }
```

```
}
```

```
[1] "STARsolo_TE"
```

```
[1] "STARsolo_TE_EM"
```

```
[1] "SoloTE_unique"
```

```
[1] "SoloTE_thr2"
```

```
[1] "SoloTE_thr1"
```

```
[1] "SoloTE_thr0"
```

```
[7]: head(overlapDfList_GENE_unique[[sample]][[tool]])
```

	Exon <chr>	gene_id <chr>	gene_name <chr>	TE <chr>	TEInSim <lgl>
A tibble: 6 × 6	0610031O16Rik	ENSMUSG00000099146.7	0610031O16Rik	B4-dup13192	FALSE
	0610040J01Rik	ENSMUSG00000060512.7	0610040J01Rik	MT2B-dup3804	FALSE
	1190002N15Rik	ENSMUSG00000045414.7	1190002N15Rik	L3b-dup1148	FALSE
	1600029O15Rik	ENSMUSG00000057818.8	1600029O15Rik	RCHARR1-dup3433	FALSE
	1700003I16Rik	ENSMUSG00000107526.1	1700003I16Rik	MERV1-I-int-dup438	FALSE
	1700016L21Rik	ENSMUSG00000101483.1	1700016L21Rik	MLT1J2-dup42	FALSE


```
[ ]: percPosOverlapTEsDf <- NULL

for (sample in samples){

  for(tool in names(objGeneList)){

    print(tool)

    df <-
    ↪cbind(melt(table(overlapDfList_GENE_unique[[sample]][[tool]]$detection,
    ↪overlapDfList_GENE_unique[[sample]][[tool]]$TEInSimulation )), tool)
    colnames(df) <- c("Detection", "ExpressedTE", "Number", "Tool")
    print(df)
    percPosOverlapTEsDf <- rbind(percPosOverlapTEsDf, df)

  }
}
```

```
[1] "STARsolo_TE"
  Detection ExpressedTE Number      Tool
1      FPs      FALSE      8 STARsolo_TE
2      TPs      FALSE    879 STARsolo_TE
3      FPs       TRUE      0 STARsolo_TE
4      TPs       TRUE     14 STARsolo_TE
[1] "STARsolo_TE_EM"
  Detection ExpressedTE Number      Tool
1      FPs      FALSE    174 STARsolo_TE_EM
2      TPs      FALSE    993 STARsolo_TE_EM
3      FPs       TRUE     21 STARsolo_TE_EM
4      TPs       TRUE     15 STARsolo_TE_EM
[1] "SoloTE_unique"
  Detection ExpressedTE Number      Tool
1      FPs      FALSE    113 SoloTE_unique
2      TPs      FALSE    953 SoloTE_unique
3      FPs       TRUE     22 SoloTE_unique
4      TPs       TRUE     18 SoloTE_unique
[1] "SoloTE_thr2"
  Detection ExpressedTE Number      Tool
1      FPs      FALSE    113 SoloTE_thr2
2      TPs      FALSE    953 SoloTE_thr2
3      FPs       TRUE     22 SoloTE_thr2
4      TPs       TRUE     18 SoloTE_thr2
[1] "SoloTE_thr1"
  Detection ExpressedTE Number      Tool
1      FPs      FALSE    113 SoloTE_thr1
2      TPs      FALSE    953 SoloTE_thr1
3      FPs       TRUE     22 SoloTE_thr1
4      TPs       TRUE     18 SoloTE_thr1
```

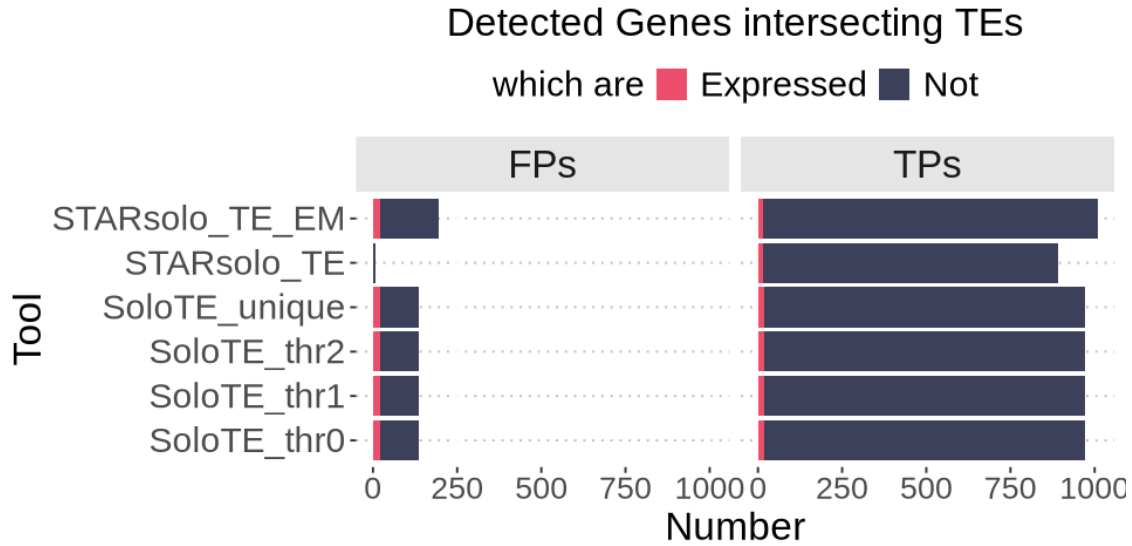
```
[1] "SoloTE_thr0"
  Detection ExpressedTE Number      Tool
1      FPs      FALSE   113 SoloTE_thr0
2      TPs      FALSE   953 SoloTE_thr0
3      FPs       TRUE    22 SoloTE_thr0
4      TPs       TRUE    18 SoloTE_thr0
```

```
[72]: percPosOverlapTEsDf$ExpressedTE <- as.character(percPosOverlapTEsDf$ExpressedTE)

options(repr.plot.width=8, repr.plot.height=4)

ggplot(percPosOverlapTEsDf, aes(y=Tool, x=Number, fill=ExpressedTE)) +
  facet_wrap(~Detection) +
  geom_col() +
  scale_fill_manual(values=c("TRUE"="#EC4D6A", "FALSE"="#3d405b"),
                    labels = c("TRUE"="Expressed", "FALSE"="Not"),
                    breaks=c("TRUE", "FALSE"), name="which are") +
  theme_pubclean() +
  ggtitle("Detected Genes intersecting TEs") +
  theme(text=element_text(size=20),
        axis.text.y = element_text(size=18),
        axis.label = element_text(size=18),
        legend.title = element_text(size=18),
        legend.text = element_text(size=18),
        strip.text = element_text(size = 20),
        strip.background = element_rect(fill="grey90"),
        plot.title = element_text(size=20, hjust=0.5))
# ggsave(paste0("figures/figures_", dataset_id, "_", sample, "/"
# ↪detectedGenesIntersectingTEs.pdf"), device="pdf", width=8, height=5)
```

```
Warning message in plot_theme(plot):
"The `axis.label` theme element is not defined in the element
hierarchy."
```

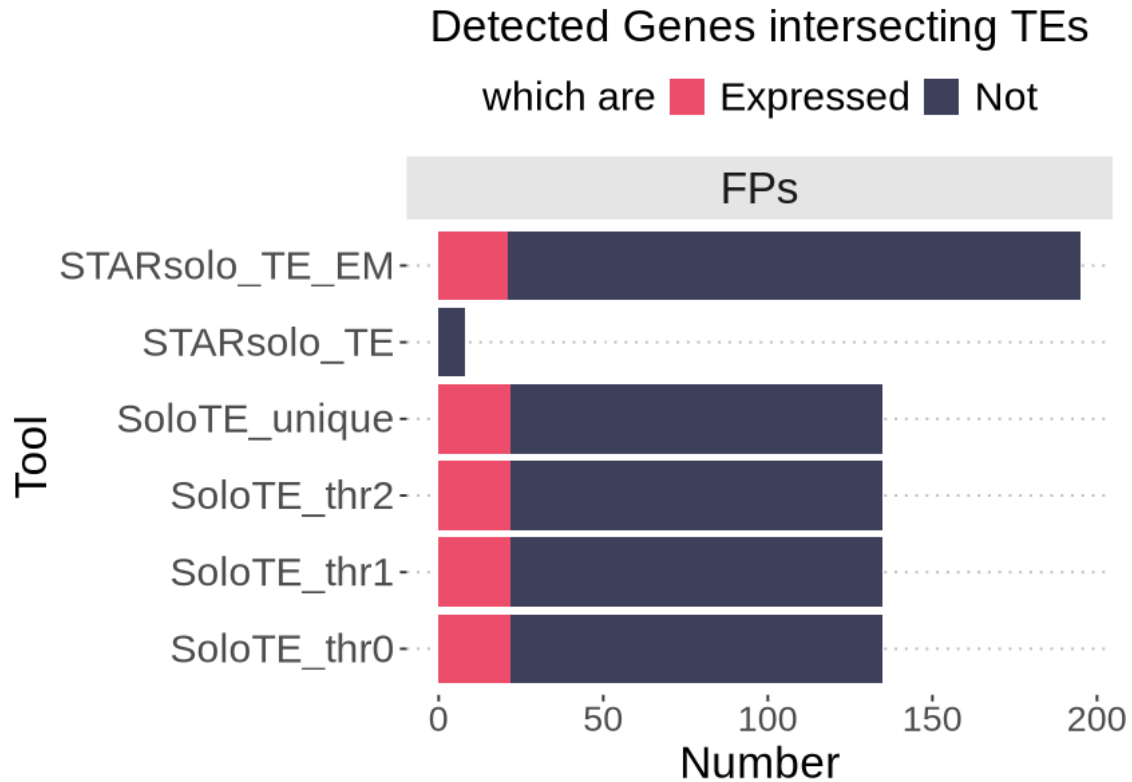


```
[73]: percPosOverlapTEsDf$ExpressedTE <- as.character(percPosOverlapTEsDf$ExpressedTE)

options(repr.plot.width=7, repr.plot.height=5)

ggplot(percPosOverlapTEsDf[percPosOverlapTEsDf$Detection=="FPs",], aes(y=Tool,
↪ x=Number, fill=ExpressedTE)) +
  facet_wrap(~Detection) +
  geom_col() +
  scale_fill_manual(values=c("TRUE"="#EC4D6A", "FALSE"="#3d405b"),
                    labels = c("TRUE"="Expressed", "FALSE"="Not"),
                    breaks=c("TRUE", "FALSE"), name="which are") +
  theme_pubclean() +
  ggtitle("Detected Genes intersecting TEs") +
  theme(text=element_text(size=20),
        axis.text.y = element_text(size=18),
        axis.label = element_text(size=18),
        legend.title = element_text(size=18),
        legend.text = element_text(size=18),
        strip.text = element_text(size = 20),
        strip.background = element_rect(fill="grey90"),
        plot.title = element_text(size=20, hjust=0.5))
# ggsave(paste0("figures/figures_", dataset_id, "_", sample, "/
↪ FPGenesIntersectingTEs.pdf"), device="pdf", width=6, height=5)
```

Warning message in plot_theme(plot):
 "The `axis.label` theme element is not defined in the element hierarchy."



5 Checkpoint

```
[74]: #save.image("workspaces/mm_RA_wGenes_evaluation_detection_afterIntersections.
      ↪Rdata")
```

```
[2]: load("workspaces/mm_RA_wGenes_evaluation_detection_afterIntersections.Rdata")
```

6 Summary figure

```
[3]: tool_order <- c(
      "SoloTE unique",
      "STARsolo",
      "STARsolo + EM",
      "Stellarscope"
    )
df_gene <- percPosOverlapGenesDf[percPosOverlapGenesDf$Tool %in%
  c("SoloTE_unique", "STARsolo_TE", "STARsolo_TE_EM", "Stellarscope"),]

df_gene_to_TE <- df_gene |>
  mutate(
```

```

ExpressedGene = as.character(ExpressedGene) == "TRUE",

Tool = recode(
  Tool,
  "SoloTE_unique" = "SoloTE unique",
  "STARsolo_TE"   = "STARsolo",
  "STARsolo_TE_EM" = "STARsolo + EM"
)
) |>

filter(
  Detection == "FPs",
  ExpressedGene
) |>

group_by(Tool) |>
summarise(
  n_errors = sum(Number),
  .groups = "drop"
) |>

mutate(
  Tool = factor(
    Tool,
    levels = tool_order
  ),

  direction = "Gene \u2192 TE",

  # Treat counts within five loci of the minimum as near-tied
  best_count = min(n_errors, na.rm = TRUE),
  top_tier = n_errors <= best_count + 5
)

df_gene_to_TE

df_TE <- percPosOverlapTEsDf[percPosOverlapTEsDf$Tool %in%
  c("SoloTE_unique", "STARsolo_TE", "STARsolo_TE_EM", "Stellarscope"),]
df_TE_to_gene <- df_TE |>
mutate(
  ExpressedTE = as.character(ExpressedTE) == "TRUE",

  Tool = recode(
    Tool,
    "SoloTE_unique" = "SoloTE unique",
    "SoloTE_thr2"   = "SoloTE max2",
    "SoloTE_thr1"   = "SoloTE max4",

```

```

    "SoloTE_thr0"      = "SoloTE multi",
    "STARsolo_TE"      = "STARsolo",
    "STARsolo_TE_EM"   = "STARsolo + EM"
  )
) |>

filter(
  Detection == "FPs",
  ExpressedTE
) |>

group_by(Tool) |>
summarise(
  n_errors = sum(Number),
  .groups = "drop"
) |>

mutate(
  direction = "TE → gene"
)
df_TE_to_gene

```

	Tool <fct>	n_errors <int>	direction <chr>	best_count <int>	top_tier <lgl>
A tibble: 4 × 5	STARsolo	25	Gene → TE	21	TRUE
	STARsolo + EM	154	Gene → TE	21	FALSE
	SoloTE unique	21	Gene → TE	21	TRUE
	Stellarscope	370	Gene → TE	21	FALSE

	Tool <chr>	n_errors <int>	direction <chr>
A tibble: 3 × 3	STARsolo	0	TE → gene
	STARsolo + EM	21	TE → gene
	SoloTE unique	22	TE → gene

```

[4]: df_misassignment <- dplyr::bind_rows(
  dplyr::select(
    df_gene_to_TE,
    Tool,
    direction,
    n_errors
  ),
  dplyr::select(
    df_TE_to_gene,
    Tool,
    direction,
    n_errors
  )
)

```

```

)
) |>
dplyr::mutate(
  Tool = factor(
    Tool,
    levels = tool_order
  ),

  direction = factor(
    direction,
    levels = c(
      "Gene → TE",
      "TE → gene"
    )
  )
) |>

dplyr::group_by(direction) |>

dplyr::mutate(
  best_count = min(
    n_errors,
    na.rm = TRUE
  ),

  top_tier = n_errors <= best_count + 5,

  # Keep zero-error observations visible
  plot_size = pmax(n_errors, 1)
) |>

dplyr::ungroup()

```

```

[9]: caption_text <- paste(
  paste0(
    "Gene → TE: false-positive TE loci overlapping expressed genes. ",
    "TE → gene: false-positive genes overlapping expressed TEs."
  ),
  paste0(
    "Black outlines indicate methods within five errors ",
    "of the lowest count in each direction."
  ),
  sep = "\n"
)

p_misassignment <- ggplot(
  df_misassignment,

```

```

aes(
  x = direction,
  y = Tool
)
) +

geom_point(
  aes(
    size = plot_size,
    fill = n_errors
  ),
  shape = 21,
  colour = "white",
  stroke = 0.4
) +

geom_point(
  data = filter(
    df_misassignment,
    top_tier
  ),
  aes(size = plot_size),
  shape = 21,
  fill = NA,
  colour = "black",
  stroke = 1.3,
  show.legend = FALSE
) +

geom_text(
  aes(label = n_errors),
  size = 3.5,
  colour = "black"
) +

scale_size_area(
  max_size = 15,
  guide = "none"
) +

scale_fill_gradient(
  low = "#FFEDAB",
  high = "#EE8F90",
  name = "Number of errors"
) +

scale_x_discrete(

```



```

labels = c(
  "Gene → TE" =
    "Gene-derived reads →\nfalse-positive TE loci",

  "TE → gene" =
    "TE-derived reads →\nfalse-positive genes"
) +

scale_y_discrete(
  limits = rev(tool_order),
  drop = TRUE
) +

labs(
  x = NULL,
  y = NULL,
  caption = caption_text
) +

theme_minimal(base_size = 14) +

theme(
  panel.grid.minor = element_blank(),

  panel.grid.major = element_line(
    colour = "grey90",
    linewidth = 0.35
  ),

  axis.text.y = element_text(
    colour = "black",
    size = 12
  ),

  axis.text.x = element_text(
    colour = "black",
    face = "bold",
    size = 11,
    lineheight = 1.1,
    margin = margin(t = 8)
  ),

  legend.position = "bottom",

  plot.caption = element_text(
    hjust = 0,

```

```

    colour = "grey30",
    size = 10,
    lineheight = 1.2,
    margin = margin(t = 14)
  ),

  plot.margin = margin(
    t = 10,
    r = 15,
    b = 15,
    l = 10
  )
)

options(repr.plot.width=7, repr.plot.height=6)
p_misassignment

ggsave("figures/summaryGeneMis.pdf", device = "pdf", width=7, height=6)

```

```

Warning message in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y,
:
"conversion failure on 'Gene-derived reads →' in 'mbcsToSbcs': dot substituted
for <e2>"
Warning message in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y,
:
"conversion failure on 'Gene-derived reads →' in 'mbcsToSbcs': dot substituted
for <86>"
Warning message in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y,
:
"conversion failure on 'Gene-derived reads →' in 'mbcsToSbcs': dot substituted
for <92>"
Warning message in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y,
:
"conversion failure on 'TE-derived reads →' in 'mbcsToSbcs': dot substituted for
<e2>"
Warning message in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y,
:
"conversion failure on 'TE-derived reads →' in 'mbcsToSbcs': dot substituted for
<86>"
Warning message in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y,
:
"conversion failure on 'TE-derived reads →' in 'mbcsToSbcs': dot substituted for
<92>"
Warning message in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y,
:
"conversion failure on 'Gene → TE: false-positive TE loci overlapping expressed
genes. TE → gene: false-positive genes overlapping expressed TEs.' in
'mbcsToSbcs': dot substituted for <e2>"

```

```

Warning message in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y,
:
"conversion failure on 'Gene → TE: false-positive TE loci overlapping expressed
genes. TE → gene: false-positive genes overlapping expressed TEs.' in
'mbcsToSbcs': dot substituted for <86>"
Warning message in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y,
:
"conversion failure on 'Gene → TE: false-positive TE loci overlapping expressed
genes. TE → gene: false-positive genes overlapping expressed TEs.' in
'mbcsToSbcs': dot substituted for <92>"
Warning message in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y,
:
"conversion failure on 'Gene → TE: false-positive TE loci overlapping expressed
genes. TE → gene: false-positive genes overlapping expressed TEs.' in
'mbcsToSbcs': dot substituted for <e2>"
Warning message in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y,
:
"conversion failure on 'Gene → TE: false-positive TE loci overlapping expressed
genes. TE → gene: false-positive genes overlapping expressed TEs.' in
'mbcsToSbcs': dot substituted for <86>"
Warning message in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y,
:
"conversion failure on 'Gene → TE: false-positive TE loci overlapping expressed
genes. TE → gene: false-positive genes overlapping expressed TEs.' in
'mbcsToSbcs': dot substituted for <92>"
Warning message in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x,
x$y, :
"conversion failure on 'Gene-derived reads →' in 'mbcsToSbcs': dot substituted
for <e2>"
Warning message in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x,
x$y, :
"conversion failure on 'Gene-derived reads →' in 'mbcsToSbcs': dot substituted
for <86>"
Warning message in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x,
x$y, :
"conversion failure on 'Gene-derived reads →' in 'mbcsToSbcs': dot substituted
for <92>"
Warning message in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x,
x$y, :
"conversion failure on 'TE-derived reads →' in 'mbcsToSbcs': dot substituted for
<e2>"
Warning message in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x,
x$y, :
"conversion failure on 'TE-derived reads →' in 'mbcsToSbcs': dot substituted for
<86>"
Warning message in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x,
x$y, :
"conversion failure on 'TE-derived reads →' in 'mbcsToSbcs': dot substituted for

```

```

<92>"
Warning message in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x,
x$y, :
"conversion failure on 'Gene → TE: false-positive TE loci overlapping expressed
genes. TE → gene: false-positive genes overlapping expressed TEs.' in
'mbcsToSbcs': dot substituted for <e2>"
Warning message in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x,
x$y, :
"conversion failure on 'Gene → TE: false-positive TE loci overlapping expressed
genes. TE → gene: false-positive genes overlapping expressed TEs.' in
'mbcsToSbcs': dot substituted for <86>"
Warning message in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x,
x$y, :
"conversion failure on 'Gene → TE: false-positive TE loci overlapping expressed
genes. TE → gene: false-positive genes overlapping expressed TEs.' in
'mbcsToSbcs': dot substituted for <92>"
Warning message in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x,
x$y, :
"conversion failure on 'Gene → TE: false-positive TE loci overlapping expressed
genes. TE → gene: false-positive genes overlapping expressed TEs.' in
'mbcsToSbcs': dot substituted for <e2>"
Warning message in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x,
x$y, :
"conversion failure on 'Gene → TE: false-positive TE loci overlapping expressed
genes. TE → gene: false-positive genes overlapping expressed TEs.' in
'mbcsToSbcs': dot substituted for <86>"
Warning message in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x,
x$y, :
"conversion failure on 'Gene → TE: false-positive TE loci overlapping expressed
genes. TE → gene: false-positive genes overlapping expressed TEs.' in
'mbcsToSbcs': dot substituted for <92>"
Warning message in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y,
:
"conversion failure on 'Gene-derived reads →' in 'mbcsToSbcs': dot substituted
for <e2>"
Warning message in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y,
:
"conversion failure on 'Gene-derived reads →' in 'mbcsToSbcs': dot substituted
for <86>"
Warning message in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y,
:
"conversion failure on 'Gene-derived reads →' in 'mbcsToSbcs': dot substituted
for <92>"
Warning message in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y,
:
"conversion failure on 'TE-derived reads →' in 'mbcsToSbcs': dot substituted for
<e2>"
Warning message in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y,

```

```

:
"conversion failure on 'TE-derived reads →' in 'mbcsToSbcs': dot substituted for
<86>"
Warning message in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y,
:
"conversion failure on 'TE-derived reads →' in 'mbcsToSbcs': dot substituted for
<92>"
Warning message in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y,
:
"conversion failure on 'Gene → TE: false-positive TE loci overlapping expressed
genes. TE → gene: false-positive genes overlapping expressed TEs.' in
'mbcsToSbcs': dot substituted for <e2>"
Warning message in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y,
:
"conversion failure on 'Gene → TE: false-positive TE loci overlapping expressed
genes. TE → gene: false-positive genes overlapping expressed TEs.' in
'mbcsToSbcs': dot substituted for <86>"
Warning message in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y,
:
"conversion failure on 'Gene → TE: false-positive TE loci overlapping expressed
genes. TE → gene: false-positive genes overlapping expressed TEs.' in
'mbcsToSbcs': dot substituted for <92>"
Warning message in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y,
:
"conversion failure on 'Gene → TE: false-positive TE loci overlapping expressed
genes. TE → gene: false-positive genes overlapping expressed TEs.' in
'mbcsToSbcs': dot substituted for <86>"
Warning message in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y,
:
"conversion failure on 'Gene → TE: false-positive TE loci overlapping expressed
genes. TE → gene: false-positive genes overlapping expressed TEs.' in
'mbcsToSbcs': dot substituted for <92>"
Warning message in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y,
:
"conversion failure on 'Gene-derived reads →' in 'mbcsToSbcs': dot substituted
for <e2>"
Warning message in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y,
:
"conversion failure on 'Gene-derived reads →' in 'mbcsToSbcs': dot substituted
for <86>"
Warning message in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y,
:
"conversion failure on 'Gene-derived reads →' in 'mbcsToSbcs': dot substituted

```

```

for <92>"
Warning message in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x,
x$y, :
"conversion failure on 'Gene-derived reads →' in 'mbcsToSbcs': dot substituted
for <e2>"
Warning message in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x,
x$y, :
"conversion failure on 'Gene-derived reads →' in 'mbcsToSbcs': dot substituted
for <86>"
Warning message in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x,
x$y, :
"conversion failure on 'Gene-derived reads →' in 'mbcsToSbcs': dot substituted
for <92>"
Warning message in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x,
x$y, :
"conversion failure on 'TE-derived reads →' in 'mbcsToSbcs': dot substituted for
<e2>"
Warning message in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x,
x$y, :
"conversion failure on 'TE-derived reads →' in 'mbcsToSbcs': dot substituted for
<86>"
Warning message in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x,
x$y, :
"conversion failure on 'TE-derived reads →' in 'mbcsToSbcs': dot substituted for
<92>"
Warning message in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x,
x$y, :
"conversion failure on 'TE-derived reads →' in 'mbcsToSbcs': dot substituted for
<e2>"
Warning message in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x,
x$y, :
"conversion failure on 'TE-derived reads →' in 'mbcsToSbcs': dot substituted for
<86>"
Warning message in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x,
x$y, :
"conversion failure on 'TE-derived reads →' in 'mbcsToSbcs': dot substituted for
<92>"
Warning message in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x,
x$y, :
"conversion failure on 'Gene → TE: false-positive TE loci overlapping expressed
genes. TE → gene: false-positive genes overlapping expressed TEs.' in
'mbcsToSbcs': dot substituted for <e2>"
Warning message in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x,
x$y, :
"conversion failure on 'Gene → TE: false-positive TE loci overlapping expressed
genes. TE → gene: false-positive genes overlapping expressed TEs.' in
'mbcsToSbcs': dot substituted for <86>"
Warning message in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x,

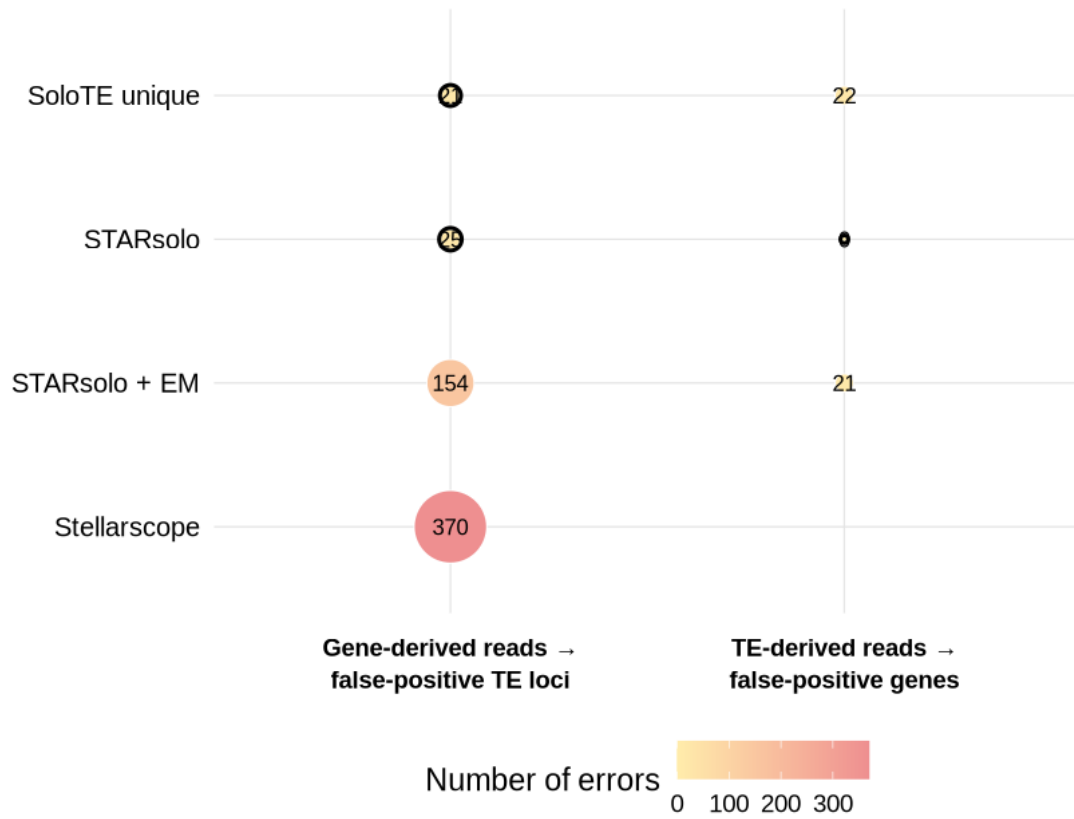
```

```

x$y, :
"conversion failure on 'Gene → TE: false-positive TE loci overlapping expressed
genes. TE → gene: false-positive genes overlapping expressed TEs.' in
'mbcsToSbcs': dot substituted for <92>"
Warning message in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x,
x$y, :
"conversion failure on 'Gene → TE: false-positive TE loci overlapping expressed
genes. TE → gene: false-positive genes overlapping expressed TEs.' in
'mbcsToSbcs': dot substituted for <e2>"
Warning message in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x,
x$y, :
"conversion failure on 'Gene → TE: false-positive TE loci overlapping expressed
genes. TE → gene: false-positive genes overlapping expressed TEs.' in
'mbcsToSbcs': dot substituted for <86>"
Warning message in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x,
x$y, :
"conversion failure on 'Gene → TE: false-positive TE loci overlapping expressed
genes. TE → gene: false-positive genes overlapping expressed TEs.' in
'mbcsToSbcs': dot substituted for <92>"
Warning message in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x,
x$y, :
"conversion failure on 'Gene → TE: false-positive TE loci overlapping expressed
genes. TE → gene: false-positive genes overlapping expressed TEs.' in
'mbcsToSbcs': dot substituted for <e2>"
Warning message in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x,
x$y, :
"conversion failure on 'Gene → TE: false-positive TE loci overlapping expressed
genes. TE → gene: false-positive genes overlapping expressed TEs.' in
'mbcsToSbcs': dot substituted for <86>"
Warning message in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x,
x$y, :
"conversion failure on 'Gene → TE: false-positive TE loci overlapping expressed
genes. TE → gene: false-positive genes overlapping expressed TEs.' in

```

'mbcsToSbcs': dot substituted for <92>"



Gene → TE: false-positive TE loci overlapping expressed genes. TE → gene: false-positive
Black outlines indicate methods within five errors of the lowest count in each direction.